FACULTY OF ARTIFICIAL INTELLIGENCE AND ENGINEERING

Trimester October/November 2025 (Term 2530)

ARR6153 Mobile Robots and Drones

Project Report

Autonomous Differential Drive Wheeled Robot

Student Name: LIAW RUO SHAN

Student ID: 1231302935

Programme: B.Sc. (Hons.) Intelligent Robotics

Table of contents

1. Introduction ---------- 3
2. Planning, Gantt Chart, and Budget Information ---------- 3
   2.1 Project Planning
   2.2 Gantt Chart
   2.3 Budget Information
3. Methodology ---------- 5
   3.1 Motor Actuation and Basic Motion
   3.2 Encoder Integration
   3.3 Manual Speed Control via User Input
   3.4 PID Speed Control
   3.5 Sensor Integration
   3.6 Wireless Communication
   3.7 Distance Estimation
4. Detailed Design ---------- 6
   4.1 Mechanical Design
   4.2 Electrical and Electronic Design
   4.3 Differential Drive Kinematics
   4.4 Software Design
5. Results ---------- 9
   5.1 Robot Prototype
   5.2 Motor Speed Control Results
   5.3 Obstacle Avoidance Results
   5.4 Distance Measurement Results
6. Discussion ---------- 11
7. Conclusion ---------- 11
8. Appendix ---------- 12

1. Introduction

Wheeled mobile robots are widely used in robotics applications because of their simple mechanical structure, efficient motion on flat surfaces, and ease of control. They are commonly applied in automation, inspection, education, and research.

This project focuses on designing and building a small autonomous differential drive wheeled robot capable of moving on a flat surface and avoiding obstacles. The robot was developed based on the concepts discussed in class, including DC motor control, pulse-width modulation (PWM), encoder feedback, PID closed-loop speed control, sensor integration, and differential drive kinematics.

The main objective of this project is to design, build, and demonstrate a compact wheeled robot that can:

- Move autonomously on a flat surface
- Control wheel speed using PWM and closed-loop PID control
- Detect and avoid obstacles using onboard sensors
- Demonstrate differential drive kinematic behaviour through independent wheel control

The robot is designed to operate within specified constraints on size, mass, and power supply, while maintaining stable and reliable motion.

2. Planning, Gantt Chart, and Budget Information

2.1 Project Planning

The robot was developed in a structured manner. Each subsystem was implemented and tested individually before full system integration. This method reduced debugging complexity, minimised integration risks, and improved overall system reliability.

The main development stages included:

1. Motor driver testing and basic motion control
2. Encoder integration and wheel speed measurement

3. Manual speed control via serial communication
4. Closed-loop PID speed control implementation
5. Sensor integration for obstacle detection
6. Behaviour logic development
7. Wireless communication via Bluetooth
8. System testing and validation

2.2 Gantt Chart

Figure 2.1 shows the overall project timeline using a Gantt chart. The chart outlines the sequence and duration of each development stage.

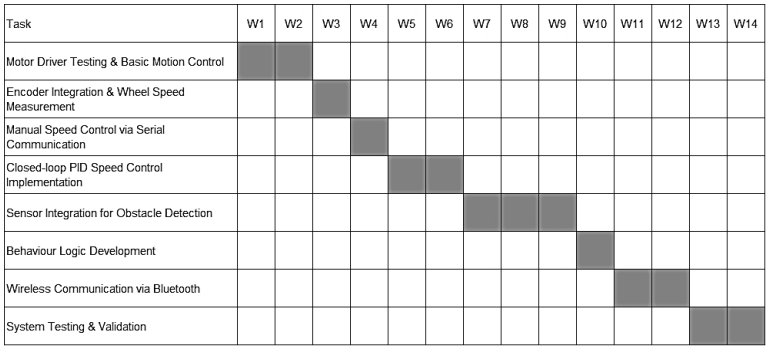

Figure 2.1 Project Gantt Chart.

2.3 Budget Information

Table 2.1 summarises the total project cost, including motors, sensors, electronics, and power supply. Low-cost, commercially available components were chosen to maintain cost efficiency.

Component	**Quantity**	**Estimated Cost**
Arduino Uno	1	RM18.00
DC Gear Motors with Encoders	2	RM70.00
L298N Motor Driver	1	RM5.90
Ultrasonic Sensor	1	RM3.30
IR Sensors	2	RM50.00
Bluetooth Module	1	RM14.90
Li-Po Battery (7.4 V)	1	RM19.90
Chassis and Wheels	-	RM19.50
Total		RM201.50

Table 2.1 Project Budget Breakdown.

3. Methodology

The robot was developed step by step, starting with basic motor actuation, sensor integration, speed control, and wireless communication. Each subsystem was tested individually before moving to the next stage.

3.1 Motor Actuation and Basic Motion

The initial development stage focused on basic motor operation. The DC gear motors were connected to an L298N dual H-bridge motor driver, which allowed the motors to move forward and backward using PWM signals and direction control pins.

3.2 Encoder Integration

In the second stage, motor encoders were integrated to measure wheel rotational speed. Encoder pulses were counted by the Arduino to obtain real-time speed feedback. This feedback is important for closed-loop speed control and differential drive movement.

3.3 Manual Speed Control via User Input

Serial communication was then implemented to allow user to manually adjust the motor speed. User-defined speed values were sent to the controller, enabling verification of encoder feedback and motor response.

3.4 PID Speed Control

To minimize speed differences between the left and right wheels and achieve straight-line motion, a closed-loop PID speed controller was implemented for each motor. Encoder feedback was used to calculate speed error, and the PID controller adjusted the PWM signal to compensate for mechanical asymmetry, friction, and load variations. This significantly improved motion stability and accuracy compared to open-loop control.

The PID control equation is:

$$u(t) = K_p e(t) + K_i \int e(t)\, dt + K_d \frac{de(t)}{dt}$$

where $e(t)$ is the wheel speed error, and K_p, K_i, and K_d are the proportional, integral, and derivative gains. The PID gains were tuned experimentally using a trial-and-error approach. The proportional gain was first increased to achieve a stable response. The integral gain was then adjusted to eliminate steady-state speed error, and a small

derivative gain was applied to reduce oscillations and improve response smoothness. This tuning process resulted in improved speed matching between two wheels and significantly reduced trajectory drift during straight-line motion.

3.5 Sensor Integration

Infrared (IR) and ultrasonic sensors were integrated for obstacle detection. Each sensor was processed using a dedicated function to handle differences in sensing range and response characteristics. Using multiple sensors improved detection reliability under varying environmental conditions.

3.6 Wireless Communication

A HC-05 Bluetooth module was added to enable wireless communication between the robot and an external device. This allowed remote control and monitoring of the robot without a physical USB connection, improving system usability during testing and demonstration.

3.7 Distance Estimation

Distance estimation was implemented using data from the motor encoders. Wheel rotation data were converted into linear displacement based on the wheel circumference, providing basic odometry information for motion analysis and performance evaluation.

$$\begin{aligned} \text{Distance} &= \text{Revolutions} \times \text{Wheel Circumference} \\ &= (\text{Speed} \times \text{Time}) \times \text{Wheel Circumference} \\ &= |\omega| \times \Delta t \times 2\pi r \end{aligned}$$

4. Detailed Design

4.1 Mechanical Design

The robot uses a differential drive configuration with two driven wheels mounted on the left and right sides of the chassis, and a passive caster wheel for balance. This configuration allows the robot to move forward and backward, turn left or right, and rotate in place by controlling the speed and direction of each wheel independently.

TT DC gear motors were selected because they are compact, low-cost, and suitable for small mobile robot applications. Each motor is equipped with an integrated encoder, which provides feedback for speed measurement and distance estimation. The chassis

was designed to securely mount all mechanical and electronic components while keeping a low centre of gravity to improve stability during motion and turning.

4.2 Electrical and Electronic Design

The robot is controlled by an Arduino Uno microcontroller, which acts as the main control unit. It reads sensor inputs, executes control algorithms, and generates control signals for the motors. Motors are controlled via the L298N dual H-bridge driver, which enables bidirectional control of the DC motors. Infrared (IR) and ultrasonic sensors are used for obstacle detection. Motor encoder pulses are processed by the Arduino for accurate speed feedback. Wireless communication is provided by a HC-05 Bluetooth module, allowing remote control and monitoring of the robot.

The system is powered by a 7.4 V lithium polymer (Li-Po) battery, which supplies sufficient current for both the motors and control electronics. The motor driver separates the high-current motor circuit from the low-power control circuitry, helping to protect the microcontroller and improve system reliability. Figure 4.1 shows the overall electrical system, highlighting power distribution and control signal flow.

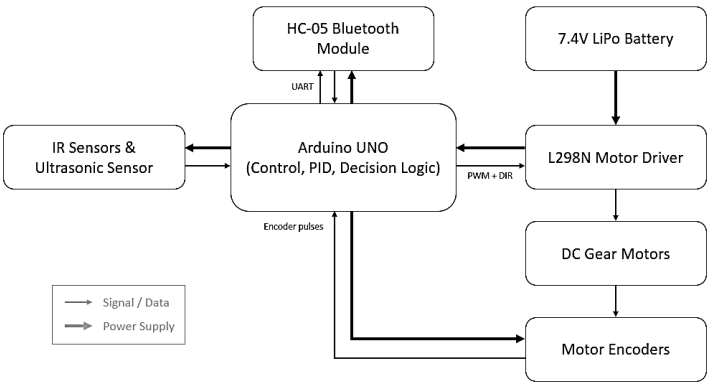

Figure 4.1 Overall System Block Diagram.

4.3 Differential Drive Kinematics

The robot is a differential drive system with two wheels of radius r, separated by a wheelbase distance L. Let ω_L and ω_R represent the angular velocities of the left and right wheels, respectively. The linear velocity v and angular velocity ω of the robot are given by:

$$v = \frac{r}{2}(\omega_R + \omega_L)$$

$$\omega = \frac{r}{L}(\omega_R - \omega_L)$$

By independently controlling the rotational speed of each wheel, the robot can move straight, follow curved paths, or rotate in place. These equations form the basis for motion control and navigation.

4.4 Software Design

The robot firmware is organized using a modular programming approach, with separate functions for motor control, sensor processing, communication, and decision-making. This structure improves code readability, simplifies debugging, and allows easier future modifications.

The main program loop continuously checks the current operating mode, reads sensor data, calculates control outputs, and updates motor commands. Two operating modes are implemented:

- **Manual mode:** User controls the robot via serial or Bluetooth commands.
- **Autonomous mode:** Robot navigates using sensor feedback.

Wheel speeds are measured using encoder feedback processed by the microcontroller. A closed-loop PID controller is implemented for each wheel to regulate speed and minimize speed differences between the left and right motors.

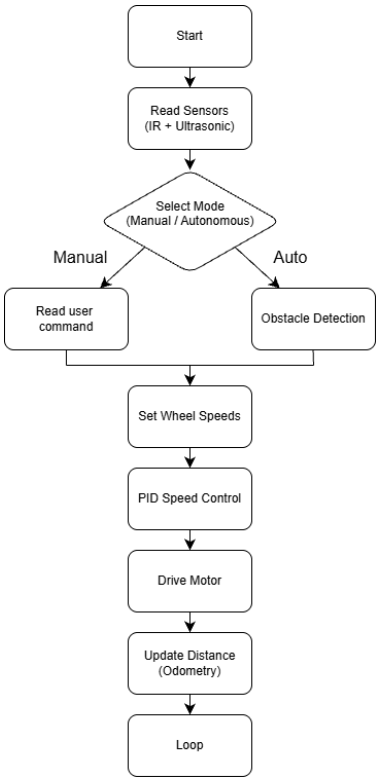

Figure 4.2 Software Control Flow of the Robot Firmware.

4.5 Arduino Source Code

The complete Arduino source code used in this project is in Appendix.

5. Results

5.1 Robot Prototype

Figures 5.1–5.3 show the front, side, and top views of the completed robot. The images highlight the placement of the main components, including the TT DC gear motors, motor encoders, infrared and ultrasonic sensors, Arduino control board, L298N motor driver, HC-05 Bluetooth module, and Li-Po battery.

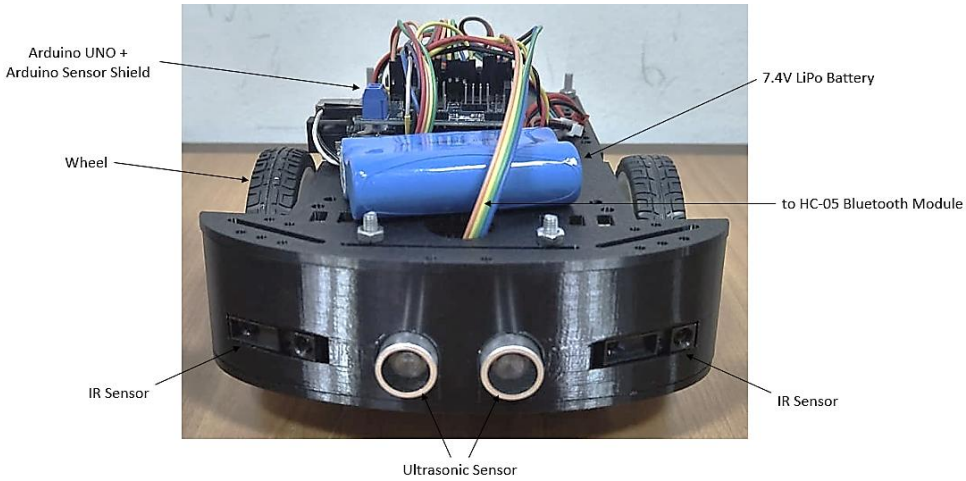

Figure 5.1 Front View of the Robot.

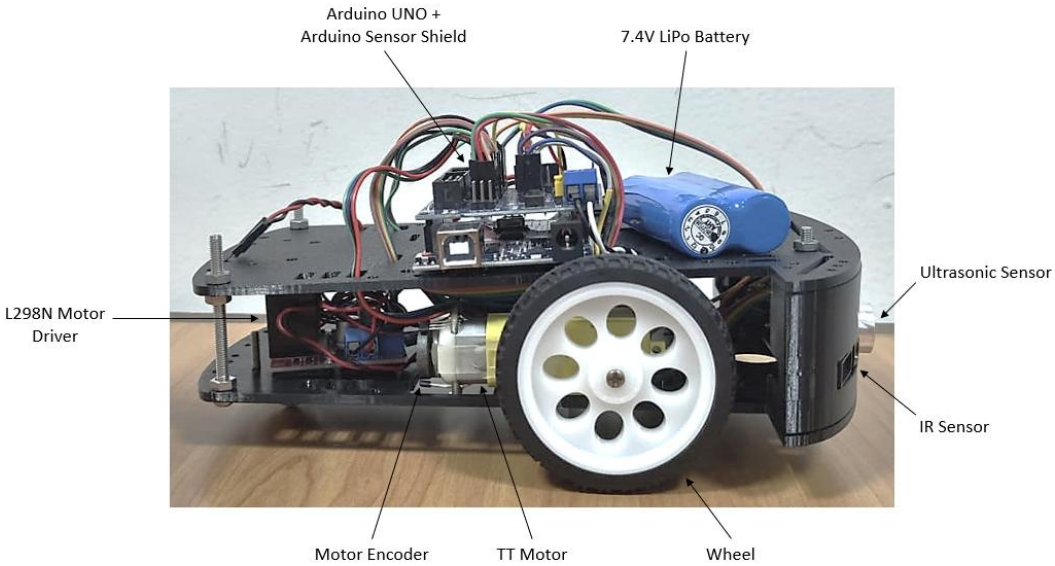

Figure 5.2 Side View of the Robot.

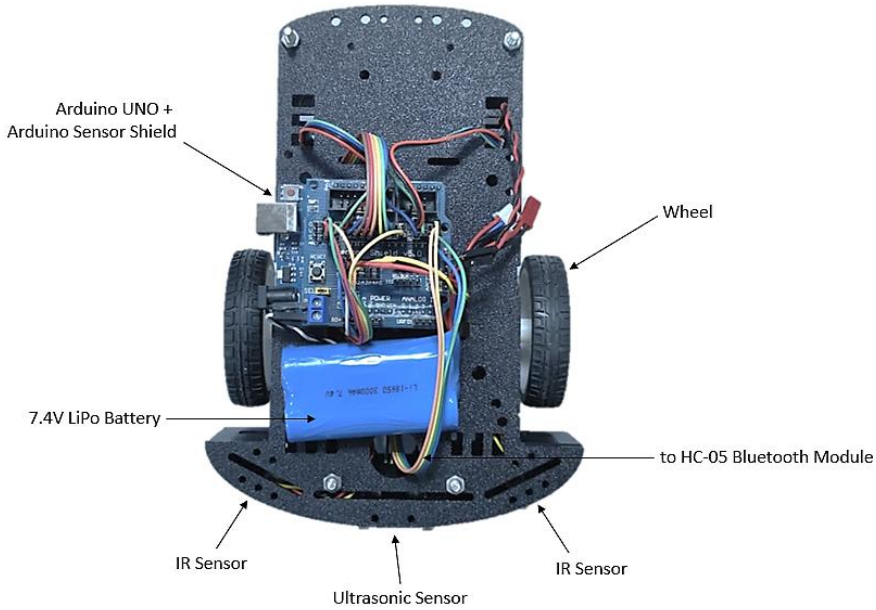

Figure 5.3 Top View of the Robot.

5.2 Motor Speed Control Results

Without PID control, the robot showed noticeable differences between the left and right wheel speeds, causing drift during straight-line motion. After implementing PID control using encoder feedback, wheel speeds were matched accurately, resulting in stable straight-line movement and significantly reduced drift.

5.3 Obstacle Avoidance Results

The IR and ultrasonic sensors successfully detected obstacles placed in front, on the left-front, and on the right-front of the robot. Based on sensor readings, the robot executed correct avoidance actions such as turning left, right, or performing U-turns. This demonstrated reliable autonomous navigation in a controlled environment.

5.4 Distance Measurement Results

Distance travelled by the robot was estimated using encoder-based odometry. The results were consistent across repeated trials, with minor errors due to wheel slip and surface irregularities. Overall, the odometry provided sufficient information for basic motion analysis and performance evaluation.

6. Discussion

During testing, the robot encountered several challenges, including speed differences between the left and right motors, sensor noise, and minor inaccuracies in time-based turning manoeuvres. The speed mismatch was corrected using encoder feedback and PID control, which greatly improved straight-line motion. Sensor noise was reduced by adjusting detection thresholds and using multiple sensors. Time-based turning introduced small angular errors due to wheel slip and surface friction, but the overall navigation remained reliable for the intended operating environment. Modular software design simplified debugging and integration. Future improvements include sensor fusion, improved odometry using gyroscopes, and systematic PID tuning methods.

7. Conclusion

This project successfully designed and built an autonomous differential drive wheeled robot within the given constraints of size, mass, and power supply. The robot can move autonomously, regulate wheel speed using closed-loop PID control, detect and avoid obstacles, and estimate distance using encoder-based odometry.

The project provided hands-on experience in mobile robot design, control, and system integration, highlighting the importance of PID tuning, sensor calibration, and systematic testing.

8. Appendix

```
#include "SoftwareSerial.h"
SoftwareSerial BTserial(0, 1); // RX(D0), TX(D1)

const byte encoder1pinA = 2;
const byte encoder1pinB = 11;
const byte encoder2pinA = 3;
const byte encoder2pinB = 12;
volatile byte encoder1PinALast;
volatile byte encoder2PinALast;

bool isTurning1 = false; // Flag to check if in "U-turn mode"
bool isTurning2 = false; // Flag to check if in "Left turn mode"
bool isTurning3 = false; // Flag to check if in "Right turn mode"
unsigned long turnStartTime = 0; // To store when the turn started
const int turnDuration1 = 3800; // Time in ms to complete 180 degrees
const int turnDuration2 = 1000; // Time in ms to complete 45 degrees

float constant_W1 = 0.0;
float constant_W2 = 0.0;
int AutoManual = 0;

int setSpeed1 = 0;
int setSpeed2 = 0;
float gf_setW1 = 0.0;
float gf_setW2 = 0.0;

float gf_PIDErrorSum1 = 0.0;
float gf_PIDErrorSum2 = 0.0;
float f_w1Error_old = 0.0;
float f_w2Error_old = 0.0;
float gf_Kd = 0.5;
float gf_Ki = 1.2;
float gf_Kp = 2.0;

#define trigPin 13 // Ultrasonic sensor
#define echoPin 4 // Ultrasonic sensor
unsigned int front_distance_mm;
unsigned int left_distance_mm;
unsigned int right_distance_mm;

volatile int gn_duration1 = 0;
volatile int gn_duration2 = 0;

float distance = 0.0; // Distance traveled

void setup()
{
  Serial.begin(9600); // Initialize the serial port
  BTserial.begin(9600); // For Bluetooth module

  pinMode(5, OUTPUT);
  pinMode(6, OUTPUT);
  pinMode(7, OUTPUT);
  pinMode(8, OUTPUT);
  pinMode(9, OUTPUT);
  pinMode(10, OUTPUT);
  pinMode(trigPin, OUTPUT); // Ultrasonic sensor
  pinMode(echoPin, INPUT); // Ultrasonic sensor

  pinMode(encoder1pinA, INPUT_PULLUP);
  pinMode(encoder1pinB, INPUT_PULLUP);
  pinMode(encoder2pinA, INPUT_PULLUP);
  pinMode(encoder2pinB, INPUT_PULLUP);

  encoder1PinALast = digitalRead(encoder1pinA);
  encoder2PinALast = digitalRead(encoder2pinA);

  attachInterrupt(digitalPinToInterrupt(encoder1pinA), wheelSpeed1, CHANGE);
  attachInterrupt(digitalPinToInterrupt(encoder2pinA), wheelSpeed2, CHANGE);
}

void loop()
```

```
{
  float f_w1, f_w2, f_w1Error, f_w2Error;
  float f_PIDout1, f_PIDout2;

  right_distance_mm = unReadIRSensor1(A0); // Read the input pin
  left_distance_mm = unReadIRSensor2(A1); // Read the input pin
  front_distance_mm = ReadUltrasonicSensor(trigPin, echoPin);

  char bytRead;

  if (Serial.available() > 0)
  {
    bytRead = Serial.read();
  }

  if (bytRead == 'm') // Manual mode
  {
    AutoManual = 1;
    distance = 0.00;
    gf_setW1 = 0.00;
    gf_setW2 = 0.00;
  }
  else if (bytRead == 'a') // Auto mode
  {
    AutoManual = 0;
    distance = 0.00;
    gf_setW1 = 0.00; // No movement before command 's'
    gf_setW2 = 0.00; // No movement before command 's'
    constant_W1 = 0.0; // No movement before command 's'
    constant_W2 = 0.0; // No movement before command 's'
  }

  if (AutoManual == 0) // Auto mode
  {
    if (bytRead == 's') // Start moving around
    {
      constant_W1 = 0.4; // Random constant rps
      constant_W2 = 0.4; // Random constant rps
    }

    if (isTurning1 == true) // U-Turn
    {
      if (millis() - turnStartTime >= turnDuration1) // Check if the time is up
      {
        isTurning1 = false; // Turn finished
        gf_setW1 = 0.0; // Stop
        gf_setW2 = 0.0;
      }
      else // Turning on spot
      {
        gf_setW1 = 0.3;
        gf_setW2 = -0.3;
      }
    }

    else if (isTurning2 == true) // Left Turn (Obstacle at right)
    {
      if (millis() - turnStartTime >= turnDuration2) // Check if the time is up
      {
        isTurning2 = false; // Turn finished
        gf_setW1 = 0.0; // Stop
        gf_setW2 = 0.0;
      }
      else // Turning on spot
      {
        gf_setW1 = 0.3;
        gf_setW2 = -0.3;
      }
    }

    else if (isTurning3 == true) // Right Turn (Obstacle at left)
    {
      if (millis() - turnStartTime >= turnDuration2) // Check if the time is up
      {
```

```
        isTurning3 = false; // Turn finished
        gf_setW1 = 0.0; // Stop
        gf_setW2 = 0.0;
      }
      else // Turning on spot
      {
        gf_setW1 = -0.3;
        gf_setW2 = 0.3;
      }
    }

    else if (isTurning1 == 0  && isTurning2 == 0 && isTurning3 == 0  && front_distance_mm <=
150) // Object detected in front
    {
      isTurning1 = true; // Start the turning sequence
      turnStartTime = millis(); // Record the current time
    }

    else if (isTurning1 == 0 && isTurning2 == 0 && isTurning3 == 0  && left_distance_mm <=
100) // Object detected at the left side
    {
      isTurning3 = true; // Start the turning sequence
      turnStartTime = millis(); // Record the current time
    }

    else if (isTurning1 == 0 && isTurning2 == 0 && isTurning3 == 0  && right_distance_mm <=
100) // Object detected at the right side
    {
      isTurning2 = true; // Start the turning sequence
      turnStartTime = millis(); // Record the current time
    }

    else // No obstacle.
    {
      gf_setW1 = constant_W1;
      gf_setW2 = constant_W2;
    }
  }

  else if (AutoManual == 1) // Manual mode
  {
    if (bytRead == '6') // Turn right
      gf_setW2 += 0.15;
    else if (bytRead == '2') // Slow down or move backward
    {
      gf_setW1 -= 0.15;
      gf_setW2 -= 0.15;
    }
    else if (bytRead == '4') // Turn left
      gf_setW1 += 0.15;
    else if (bytRead == '5') // Stop
    {
      gf_setW1 = 0.00;
      gf_setW2 = 0.00;
    }
    else if (bytRead == '8') // Move forward
    {
      gf_setW1 += 0.15;
      gf_setW2 += 0.15;
    }
    if (front_distance_mm <= 100 || left_distance_mm <= 100 || right_distance_mm <= 100)
    {
      gf_setW1 = 0.0;
      gf_setW2 = 0.0;
    }
  }

  noInterrupts();
  // Pulse1
  f_w1 = (gn_duration1/960.0)*10.0;
  gn_duration1 = 0;

  // Pulse2
  f_w2 = (gn_duration2/960.0)*10.0;
```

```
  gn_duration2 = 0;
  interrupts();

  f_w1Error = gf_setW1 - f_w1;
  f_w2Error = gf_setW2 - f_w2;

  gf_PIDErrorSum1 = gf_Ki*f_w1Error + gf_PIDErrorSum1;
  gf_PIDErrorSum2 = gf_Ki*f_w2Error + gf_PIDErrorSum2;
  f_PIDout1 = gf_Kp*f_w1Error + gf_Kd*(f_w1Error - f_w1Error_old) + gf_PIDErrorSum1;
  f_PIDout2 = gf_Kp*f_w2Error + gf_Kd*(f_w2Error - f_w2Error_old) + gf_PIDErrorSum2;
  setSpeed1 = f_PIDout1*960.0/10.0;
  setSpeed2 = f_PIDout2*960.0/10.0;

  f_w1Error_old = f_w1Error;
  f_w2Error_old = f_w2Error;

  SetMotor1(setSpeed1);
  SetMotor2(setSpeed2);

  if (AutoManual == 0)
  {
    Serial.print(left_distance_mm); // Obstacle distance
    Serial.print("; ");

    Serial.print(right_distance_mm); // Obstacle distance
    Serial.print("; ");

    Serial.print(front_distance_mm); // Obstacle distance
    Serial.print("; ");
  }

  /* Serial.print(f_w1);
  Serial.print(" ");
  Serial.println(f_w2); */

  float d1 = fabs(f_w1) * 0.1 * 2.0 * PI * 0.03;
  float d2 = fabs(f_w2) * 0.1 * 2.0 * PI * 0.03;

  float d = (d1 + d2) / 2.0;  // Robot traveled distance
  distance += d;
  Serial.print(distance);
  Serial.println("m");

  delay(100);
}

void wheelSpeed1()
{
  byte Lstate = digitalRead(encoder1pinA);
  if((encoder1PinALast == LOW) && Lstate==HIGH)
  {
    byte Bstate = digitalRead(encoder1pinB);
    if(Bstate == LOW)
    {
      gn_duration1--; // Forward
    }
    else
    {
      gn_duration1++; // Reverse
    }
  }
  encoder1PinALast = Lstate;
}

void wheelSpeed2()
{
  byte Lstate = digitalRead(encoder2pinA);
  if((encoder2PinALast == LOW) && Lstate==HIGH)
  {
    byte Bstate = digitalRead(encoder2pinB);
    if(Bstate == LOW)
    {
      gn_duration2--; // Forward
    }
```

```
    else
    {
      gn_duration2++; // Reverse
    }
  }
  encoder2PinALast = Lstate;
}

void SetMotor1(int nSpeed)
{
  if (nSpeed >= 0)
  {
    analogWrite(5, nSpeed);
    digitalWrite(7, LOW);
    digitalWrite(8, HIGH);
  }
  else // nSpeed is negative.
  {
    analogWrite(5, -nSpeed);
    digitalWrite(7, HIGH);
    digitalWrite(8, LOW);
  }
}

void SetMotor2(int nSpeed)
{
  if (nSpeed >= 0)
  {
    analogWrite(6, nSpeed);
    digitalWrite(9, LOW);
    digitalWrite(10, HIGH);
  }
  else // nSpeed is negative.
  {
    analogWrite(6, -nSpeed);
    digitalWrite(9, HIGH);
    digitalWrite(10, LOW);
  }
}

unsigned int unReadIRSensor1(unsigned int unPin) // 10cm - 80cm
{
  if (unPin < A0 || unPin > A5) {
    return 0; // Return 0 to indicate invalid pin
  }

  int value = analogRead(unPin); // Read Sensor
  float volts = value * 0.0048828125; // Convert to Voltage
  if (volts < 0.1)
    volts = 0.1;  // Prevent math explosion

  unsigned int distance_mm = 262.22 * pow(volts, -1.299);

  // Limit the range (100mm - 8000mm)
  if (distance_mm < 100) {
    distance_mm = 100;
  }
  else if (distance_mm > 8000) {
    distance_mm = 8000;
  }

  return distance_mm;
}

unsigned int unReadIRSensor2(unsigned int unPin) // 4cm - 30cm
{
  if (unPin < A0 || unPin > A5) {
    return 0; // Return 0 to indicate invalid pin
  }

  int value = analogRead(unPin); // Read Sensor
  float volts = value * 0.0048828125; // Convert to Voltage
  if (volts < 0.1)
    volts = 0.1;  // Prevent math explosion
```

```
  unsigned int distance_mm = 123.99 * pow(volts, -1.061);

  // Limit the range (40mm - 300mm)
  if (distance_mm < 40) {
    distance_mm = 40;
  }
  else if (distance_mm > 300) {
    distance_mm = 300;
  }

  return distance_mm;
}

unsigned int ReadUltrasonicSensor(byte trig, byte echo)
{
  unsigned long duration, distance;

  digitalWrite(trig, LOW);
  delayMicroseconds(2);
  digitalWrite(trig, HIGH);
  delayMicroseconds(10);
  digitalWrite(trig, LOW);

  duration = pulseIn(echo, HIGH, 25000);
  if (duration == 0)
    duration = 8000; // Assume no obstacle

  distance = duration*0.1718;

  return distance;
}
```